Faculty of Engineering & Technology

Electrical & Computer Engineering Department

Operating Systems – ENCS3390

Project One Report

Prepared by:

Name: Nagham Massis. **Number: 1220149.**

Instructor: Yazan Abu Farha.

Date: 8/12/2024

Abstract:

This project focuses on analyzing word frequency in a large dataset to find the top 10 most frequent words using three methods: naive, multiprocessing, and multithreading. The naive method handles the dataset sequentially, while multiprocessing divides the work among multiple processes, and multithreading splits the task across multiple threads. The performance of each method is compared by measuring execution times, and their scalability is evaluated using Amdahl's Law. The results show that multiprocessing and multithreading are much faster than the naive method, highlighting the efficiency and scalability of parallel computing for large-scale text analysis.

Contents

Abstract: I

Table of figures: III

List of tables: IV

Environment: 1

How I achieved multiprocessing requirements & The API and functions that I used: 2

How I achieved multithreading requirements & The API and functions that I used: 4

Analysis according to Amdahl's law: 5

- **1.Naïve Approach:** 5
- **2.Multiprocessing Approach:** 6
- **3.Multithreading Approach:** 7

Performance: 8

Conclusion: 12

Table of figures:

Figure 1: Naive approach result. 9
Figure 2: 2 processes result. 9
Figure 3: 4 processes result. 9
Figure 4: 6 processes result. 10
Figure 5: 8 processes result. 10
Figure 6: 2 threads result. 10
Figure 7: 4 threads result. 11
Figure 8: 6 threads result. 11
Figure 9: 8 threads result. 11

List of tables:

Table 1: multiprocessing table. 6
Table 2: multithreading table..... 7
Table 3: performance table. 8

Environment:

- **Computer specifics:**

1. **Processor**: 11th Gen Intel® Core™ i7-1165G7.
    - **Cores and Threads**: 4 cores, 8 threads.
    - **Speed**: 2.80 GHz.
2. **Memory (RAM)**: 16.0 GB

- **Operating System:** Windows 11 Pro 64-bit (Build 22631).
- **Programming Language:** C.
- **IDE:** The Ubuntu terminal.
- **I didn't use virtual machine.**

How I achieved multiprocessing requirements & The API and functions that I used:

I used multiprocessing to speed up word frequency analysis by splitting the task across multiple processes. Each process handles a portion of the dataset, and shared memory is used to store and update word frequencies. A semaphore is used for synchronization to prevent race conditions, allowing for parallel execution across multiple CPU cores.

Key Steps:

1. **Process Creation**: The program creates child processes using fork(), with each process handling a subset of the word list.
2. **Shared Data and Synchronization**: Shared memory is used to store word frequencies, and a semaphore ensures that only one process modifies the shared data at a time, preventing race conditions.
3. **Process Execution**: Each child process counts the frequency of words in its assigned portion of the dataset.
4. **Waiting for Processes**: The parent process waits for all child processes to finish using wait() before proceeding with sorting and printing the results.

API and Functions Used for Multiprocessing:

1. **fork()**: Creates child processes, with each process performing a portion of the word frequency analysis.
2. **mmap()**: Allocates shared memory, allowing multiple processes to access and modify the same data (word list and frequencies).
3. **sem_init(), sem_wait(), sem_post(), sem_destroy()**: Functions from the semaphore API used to synchronize access to shared data, ensuring only one process modifies the data at a time.

4. **wait()**: The parent process waits for all child processes to finish before proceeding with sorting and displaying results.

In summary, multiprocessing is implemented using the fork system call, shared memory, and semaphores to enable efficient word frequency analysis by splitting the task across multiple processes running in parallel.

How I achieved multithreading requirements & The API and functions that I used:

I used multithreading to parallelize the word frequency analysis by splitting the task across multiple threads. Each thread processes a portion of the dataset, speeding up the computation.

Key Steps:

1. **Thread Creation**: The program creates threads using pthread_create(), where each thread processes a subset of the word list.
2. **Shared Data and Synchronization**: Threads access shared data (arrWord for word frequencies) and use a mutex (pthread_mutex_t) to ensure thread-safe access, preventing race conditions.
3. **Thread Execution**: Each thread processes a portion of the words, counts their frequency, and updates the shared word frequency data.
4. **Waiting for Threads**: After all threads are created, the main thread waits for their completion using pthread_join() before proceeding with sorting and printing the results.

API and Functions Used:

1. **pthread_create()**: Creates threads for parallel execution.
2. **pthread_join()**: Ensures the main thread waits for all threads to finish before continuing.
3. **pthread_mutex_init(), pthread_mutex_lock(), pthread_mutex_unlock(), pthread_mutex_destroy()**: These functions initialize and manage the mutex, ensuring synchronized access to shared data.

In summary, multithreading is implemented using the pthread library, which allows the program to perform word frequency analysis in parallel across multiple threads, improving performance.

Analysis according to Amdahl's law:

1. Naïve Approach:

- **Total Time = 716.362284 seconds.**
- **Serial Time = 716.362239 seconds.**
- **Serial Portion =** $(\text{serial time} / \text{total time}) \times 100\%$

  = (716.362239 / 716.362284) ×100%

  = 99.9999937% (No parallelization).
- **Maximum speedup =** $\frac{1}{S+\frac{1-S}{N}} = \frac{1}{0.999999937+\frac{0.000000063}{1}} = 1$
- The analysis of the naïve approach using Amdahl's Law shows that most of the time, about 99.99%, is spent on tasks that can't be parallelized, like reading the file, counting word frequency and sorting. This means that the speedup from parallelization is limited because these tasks need to be done in sequence. With only one core, the maximum speedup is 1. Therefore, even though parallelization could help, the naïve approach doesn't see much improvement in performance because most of it is serial.

2. Multiprocessing Approach:

- **Serial Portion** = (serial time / total time) $\times 100\%$
- **Maximum speedup** = $\frac{1}{S+\frac{1-S}{N}}$

Number of processes	Max speedup
2 processes	1.99
4 processes	3.92
6 processes	5.8
8 processes	7.6

Table 1: multiprocessing table.

- In the multiprocessing approach using 4 cores, Amdahl's Law analysis shows that the serial part of the code, like reading data, sorting and printing most 10 frequent words, takes up about 0.632% of the total time. This is a big improvement compared to the naïve approach, showing the benefits of parallelization. However, even with multiple cores, the maximum speedup is only about 3.925. This is because some tasks are still done sequentially, and there is overhead from managing parallel processes. While multiprocessing helps, more optimization may be needed to fully use the available resources.
- The optimal number pf child processes is 8 processes.

3. Multithreading Approach:

- **Serial Portion = (serial time / total time) ×100%**
- **Maximum speedup =** $\frac{1}{S+\frac{1-S}{N}} = \frac{1}{0.00552+\frac{0.99448}{4}}$ **= 3.934**

Number of threads	Max speedup
2 threads	1.99
4 threads	3.93
6 threads	5.82
8 threads	7.63

Table 2: multithreading table.

- In the multithreading approach using 4 cores, Amdahl's Law analysis shows that the serial part of the code, like reading data, sorting and printing most 10 frequent words, takes up about 0.552% of the total time. This is a big improvement compared to the naïve approach, showing the benefits of parallelization. However, even with multiple cores, the maximum speedup is only about 3.934. This is because some tasks are still done sequentially, and there is overhead from managing parallel processes. While multithreading helps, more optimization may be needed to fully use the available resources.
- The optimal number pf child threads is 8 threads.

Performance:

In our project, which involved counting frequency using different parallel computing approaches, we assessed the performance of three methods: naïve sequential processing, multiprocessing, multithreading. After thorough evaluation, we rank these approaches based on their execution time.

Approach Type	Total Time (sec)	Serial Time (sec)
Naïve	716.362239	716.362284
2 processes	485.682599	1.606450
4 processes	256.020707	1.619808
6 processes	234.736552	1.598196
8 processes	218.150678	1.588897
2 threads	491.865982	1.494383
4 threads	268.855453	1.484936
6 threads	244.024853	1.476971
8 threads	216.314461	1.475403

Table 3: performance table.

Naïve approach: The naive approach for counting word frequency performs well on smaller datasets due to its simplicity, but it can be slow for larger datasets. Its performance is limited because it needs to check every word against all previously seen words. As the dataset grows, this becomes less efficient. Despite this, the naive method can still be effective when speed isn't a critical factor or when the dataset is small enough that the extra processing time is minimal.

Multiprocessing approach: The multiprocessing approach speeds up word frequency counting by using multiple CPU cores to process data in parallel. This reduces the time for large datasets, though some overhead is introduced when merging results. It’s more efficient than the naive approach for big datasets but may not provide much benefit for smaller ones.

Multithreading approach: The multithreading for word frequency counting can improve performance by processing data in parallel across multiple cores. Each thread works on a different portion of the dataset, speeding up the task. However, the performance gain depends on the number of CPU cores and the overhead from managing shared resources. If there are more

threads than cores, the benefits of parallelism may be reduced. Despite this, multithreading can still provide a significant speedup for large datasets.

```
codebind@codebind-HP-Laptop-15-dw3xxx:~/Desktop/OSProject$ gcc -o naive naive.c
codebind@codebind-HP-Laptop-15-dw3xxx:~/Desktop/OSProject$ ./naive
File 'text8.txt' opened.
Top 10 Most Frequent Words:
1. the - 1061396
2. of - 593677
3. and - 416629
4. one - 411764
5. in - 372201
6. a - 325873
7. to - 316376
8. zero - 264975
9. nine - 250430
10. two - 192644
Total time: 716.362284 seconds
Serial time(Reading + Counting words + Sorting + Printing): 716.362239 seconds
```

Figure 1: Naive approach result.

```
codebind@codebind-HP-Laptop-15-dw3xxx:~/Desktop/OSProject$ gcc -o multiprocessing multiprocessing.c
codebind@codebind-HP-Laptop-15-dw3xxx:~/Desktop/OSProject$ ./multiprocessing
File 'text8.txt' opened.
Top 10 Most Frequent Words:
1. the - 1061396
2. of - 593677
3. and - 416629
4. one - 411764
5. in - 372201
6. a - 325873
7. to - 316376
8. zero - 264975
9. nine - 250430
10. two - 192644
Total time: 485.682599 seconds
Serial time(Reading + Sorting + Printing): 1.606450 seconds
Parallel time(Counting words): 484.076076 seconds
```

Figure 2: 2 processes result.

```
codebind@codebind-HP-Laptop-15-dw3xxx:~/Desktop/OSProject$ gcc -o multiprocessing multiprocessing.c
codebind@codebind-HP-Laptop-15-dw3xxx:~/Desktop/OSProject$ ./multiprocessing
File 'text8.txt' opened.
Top 10 Most Frequent Words:
1. the - 1061396
2. of - 593677
3. and - 416629
4. one - 411764
5. in - 372201
6. a - 325873
7. to - 316376
8. zero - 264975
9. nine - 250430
10. two - 192644
Total time: 256.020707 seconds
Serial time(Reading + Sorting + Printing): 1.619808 seconds
Parallel time(Counting words): 254.400848 seconds
```

Figure 3: 4 processes result.

```
codebind@codebind-HP-Laptop-15-dw3xxx:~/Desktop/OSProject$ gcc -o multiprocessing multiprocessing.c
codebind@codebind-HP-Laptop-15-dw3xxx:~/Desktop/OSProject$ ./multiprocessing
File 'text8.txt' opened.
Top 10 Most Frequent Words:
1. the - 1061396
2. of - 593677
3. and - 416629
4. one - 411764
5. in - 372201
6. a - 325873
7. to - 316376
8. zero - 264975
9. nine - 250430
10. two - 192644
Total time: 234.736552 seconds
Serial time(Reading + Sorting + Printing): 1.598196 seconds
Parallel time(Counting words): 233.138275 seconds
```

Figure 4: 6 processes result.

```
codebind@codebind-HP-Laptop-15-dw3xxx:~/Desktop/OSProject$ gcc -o multiprocessing multiprocessing.c
codebind@codebind-HP-Laptop-15-dw3xxx:~/Desktop/OSProject$ ./multiprocessing
File 'text8.txt' opened.
Top 10 Most Frequent Words:
1. the - 1061396
2. of - 593677
3. and - 416629
4. one - 411764
5. in - 372201
6. a - 325873
7. to - 316376
8. zero - 264975
9. nine - 250430
10. two - 192644
Total time: 218.150678 seconds
Serial time(Reading + Sorting + Printing): 1.588897 seconds
Parallel time(Counting words): 216.561729 seconds
```

Figure 5: 8 processes result.

```
codebind@codebind-HP-Laptop-15-dw3xxx:~/Desktop/OSProject$ gcc -o multithreading multithreading.c -pthread
codebind@codebind-HP-Laptop-15-dw3xxx:~/Desktop/OSProject$ ./multithreading
File 'text8.txt' opened.
Top 10 Most Frequent Words:
1. the - 1061396
2. of - 593677
3. and - 416629
4. one - 411764
5. in - 372201
6. a - 325873
7. to - 316376
8. zero - 264975
9. nine - 250430
10. two - 192644
Total time: 491.865982 seconds
Serial time(Reading + Sorting + Printing): 1.494383 seconds
Parallel time(Counting words): 490.371538 seconds
```

Figure 6: 2 threads result.

```
codebind@codebind-HP-Laptop-15-dw3xxx:~/Desktop/OSProject$ gcc -o multithreading multithreading.c -pthread
codebind@codebind-HP-Laptop-15-dw3xxx:~/Desktop/OSProject$ ./multithreading
File 'text8.txt' opened.
Top 10 Most Frequent Words:
1. the - 1061396
2. of - 593677
3. and - 416629
4. one - 411764
5. in - 372201
6. a - 325873
7. to - 316376
8. zero - 264975
9. nine - 250430
10. two - 192644
Total time: 268.855453 seconds
Serial time(Reading + Sorting + Printing): 1.484936 seconds
Parallel time(Counting words): 267.370476 seconds
```

Figure 7: 4 threads result.

```
codebind@codebind-HP-Laptop-15-dw3xxx:~/Desktop/OSProject$ gcc -o multithreading multithreading.c -pthread
codebind@codebind-HP-Laptop-15-dw3xxx:~/Desktop/OSProject$ ./multithreading
File 'text8.txt' opened.
Top 10 Most Frequent Words:
1. the - 1061396
2. of - 593677
3. and - 416629
4. one - 411764
5. in - 372201
6. a - 325873
7. to - 316376
8. zero - 264975
9. nine - 250430
10. two - 192644
Total time: 244.024853 seconds
Serial time(Reading + Sorting + Printing): 1.476971 seconds
Parallel time(Counting words): 242.547840 seconds
```

Figure 8: 6 threads result.

```
codebind@codebind-HP-Laptop-15-dw3xxx:~/Desktop/OSProject$ gcc -o multithreading multithreading.c -pthread
codebind@codebind-HP-Laptop-15-dw3xxx:~/Desktop/OSProject$ ./multithreading
File 'text8.txt' opened.
Top 10 Most Frequent Words:
1. the - 1061396
2. of - 593677
3. and - 416629
4. one - 411764
5. in - 372201
6. a - 325873
7. to - 316376
8. zero - 264975
9. nine - 250430
10. two - 192644
Total time: 216.314461 seconds
Serial time(Reading + Sorting + Printing): 1.475403 seconds
Parallel time(Counting words): 214.839018 seconds
```

Figure 9: 8 threads result.

Conclusion:

In conclusion, this project shows that multiprocessing and multithreading are much faster than the naive sequential method. While the naive method is simple, it doesn't perform well with large datasets because it processes everything sequentially. On the other hand, multiprocessing and multithreading speed up the process by dividing the work into smaller tasks handled by multiple processes or threads. Amdahl's Law helps us understand that although adding more processes or threads improves performance, the gains become smaller after a certain point. Overall, parallel computing is very effective for tasks like word frequency analysis, making it faster and more efficient.